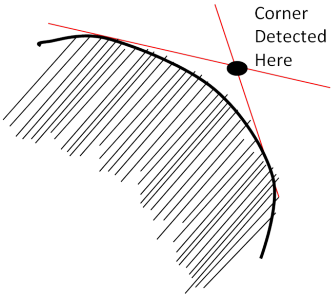
Harris Corners and Feature Matching

Dr. Khalil Yousef

Computer Science and Engineering Department
Qatar University (QU)

The Harris Corner Detector

The Harris Corner Detector



- Before SIFT came along, this was probably the world's most popular interest point detector.
- It serves a useful purpose even today if your application **does NOT require scale-space analysis** or the value of **scale** σ is **fixed** and known in advance.

The Harris Corner Detector

- By a **corner** we mean any pixel in the vicinity of which the gray levels show significant variations in at least two different directions.
- The secret of a good corner detector is that its characterization of a pixel as a corner should be invariant:
  - to in-place rotations of the image
  - as long as there does **not** exist a direction with **no** gray level variations inside the window to which the operator is applied.
- The Harris corner detector possesses this invariance.

The Harris Corner Detector

- The very first step is to compute the **gradient** at each point in the image.
- Let d_x estimates the $x-$derivative and d_y the $y-$derivative at a pixel at scale σ.
  - $d_x$and d_y can be the outputs of the x and y- oriented Haar filters at scale σ
    - **Haar filters are used to compute first-order derivatives and are discussed in the next slide.**
  - If scale is not important, d_x and d_y can be the $x-$ and $y-$components of the local gradient as measured by the Sobel operator.

Aside - Harr Wavelet Filters:
Used to compute first-order derivatives d_x and d_y

- The basic form of the **Harr wavelet** is $\begin{bmatrix} -1 & 1 \end{bmatrix}$ along x and $\begin{bmatrix} 1 \\ -1 \end{bmatrix}$ along y. These forms are scaled up to $M \times M$ operator where M is the smallest even integer greater than $4 \times \sigma$.
- Shown below are 6×6 operators for $\sigma = 1.2$. note that σ in the size of the operators refers to the scale at which the interest point was detected. We use d_x to denote the output of the $\frac{\partial}{\partial x}$ operator and d_y to denote the output for $\frac{\partial}{\partial y}$.

$\frac{\partial}{\partial x}$ for σ=1.2

$$\begin{bmatrix} -1 & -1 & -1 & 1 & 1 & 1 \\ -1 & -1 & -1 & 1 & 1 & 1 \\ -1 & -1 & -1 & 1 & 1 & 1 \\ -1 & -1 & -1 & 1 & 1 & 1 \\ -1 & -1 & -1 & 1 & 1 & 1 \\ -1 & -1 & -1 & 1 & 1 & 1 \end{bmatrix}$$

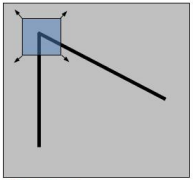
,

$\frac{\partial}{\partial y}$ for σ=1.2

$$\begin{bmatrix} 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 & 1 \\ -1 & -1 & -1 & -1 & -1 & -1 \\ -1 & -1 & -1 & -1 & -1 & -1 \\ -1 & -1 & -1 & -1 & -1 & -1 \end{bmatrix}$$

The Harris Corner Detector

- We now construct the following matrix in a $5\sigma \times 5\sigma$ neighborhood of the pixel where we wish to determine the presence or the absence of a corner:
  $C = \begin{bmatrix} \sum d_x^2 & \sum d_x d_y \\ \sum d_x d_y & \sum d_y^2 \end{bmatrix}$ where the summations are over all the pixels in the $5\sigma \times 5\sigma$ neighborhood.
  - This 2×2 matrix has full rank at **a genuine corner**. However, if the pixel on which the $5\sigma \times 5\sigma$ window is centered is on a straightedge (with all other pixels possessing uniform gray levels), its rank will reduce to 1.



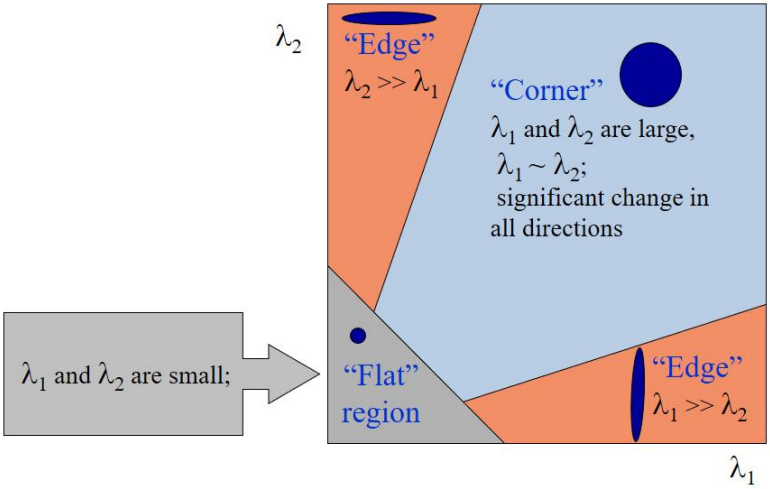
"corner":
significant change
in all directions

The Harris Corner Detector

- Perform **Eigen-Decomposition** on C, although no need to actually compute it as discussed below.
- Let λ_1 and λ_2 be the two eigenvalues of C. We assume $\lambda_1 \geq \lambda_2$.
  - We place a threshold on the ratio $r = \lambda_2/\lambda_1$, and typically
  - require $r \gtrsim 0.1$ for the pixel at the center of the window to be called a corner.
    - The ratio r can be computed very efficiently **without** an explicit eigen-decomposition of C
    - Note: this is due to the fact that $Tr(C) = \sum d_x^2 + \sum d_y^2 = \lambda_1 + \lambda_2$ and $det(C) = \sum d_x^2 \sum d_y^2 - (\sum d_x d_y)^2 = \lambda_1 \lambda_2$.
    - This implies $\frac{det(C)}{[Tr(C)]^2} = \frac{\lambda_1 \lambda_2}{(\lambda_1+\lambda_2)^2} = \frac{r}{(1+r)^2}$.
    - Therefore, the ratio $\frac{det(C)}{[Tr(C)]^2}$ can be directly thresholded for corner detection.

Aside: Interpreting the eigenvalues

Classification of image points using eigenvalues of C:



Harris Features Example (in red)

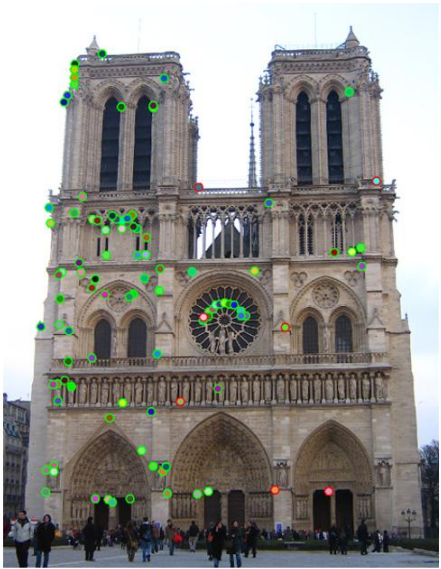


Possible Harris Descriptor

- When the low-level features extracted from the two images are Harris corners, we can establish correspondences by directly comparing the gray levels in, say, a $(M+1) \times (M+1)$ window around the corner pixel in one image with the gray levels in a similar window around the corresponding pixel in the other image.
  - We can use the following two metrics to judge similarity:
    - SSD: Sum of squared difference $= \sum_i \sum_j |f_1(i,j) - f_2(i,j)|^2$
    - NCC: Normalized cross correlation $= \frac{\sum\sum(f_1(i,j)-m_1)(f_2(i,j)-m_2)}{\sqrt{[\sum\sum(f_1(i,j)-m_1)^2][\sum\sum(f_2(i,j)-m_2)^2]}}$
    - Where: f_1: Image 1, f_2 : Image 2, m_1 : mean of f_1 , m_2 : mean of f_2
- Example:
  - See the following link:
    `https://docs.opencv.org/4.x/dc/d0d/tutorial_py_features_harris.html`

Feature Matching or Establishing Correspondences Between Image Pairs

Which features match?



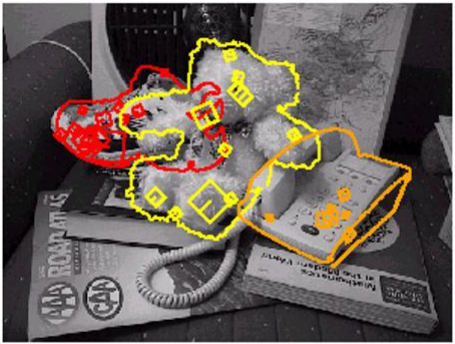


Applications

- Features are used for:
  - **Image alignment (e.g., mosaics)**
  - 3D reconstruction
  - Motion tracking
  - **Object recognition**
  - Indexing and database retrieval
  - Robot navigation
  - Others

Object Recognition Example: (David Lowe)



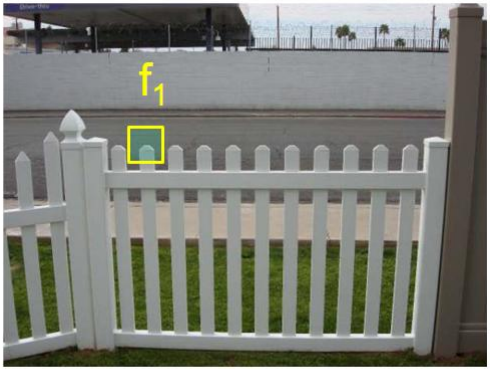


Feature Matching: Basic Idea

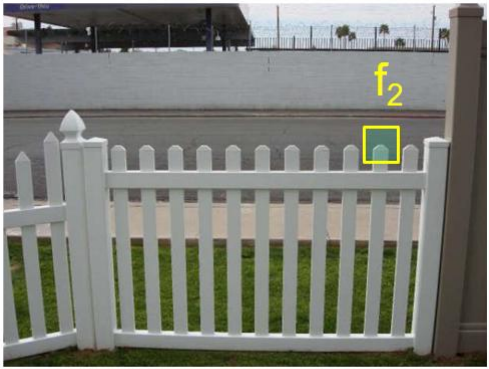
- Given a feature in I_1, how to find the best match in I_2?
  - Define distance function that compares two descriptors
  - Test all the features in I_2, find the one with min distance

Feature Distance

- How to define the difference between two features descriptor vectors f_1, f_2?
  - Simple approach: L2 distance or Euclidean distance , $||f_1 - f_2||$
  - Can give small distances for ambiguous (incorrect) matches.



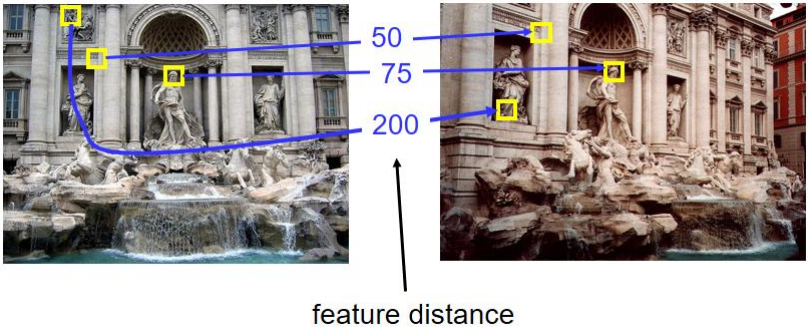
I_1



I_2

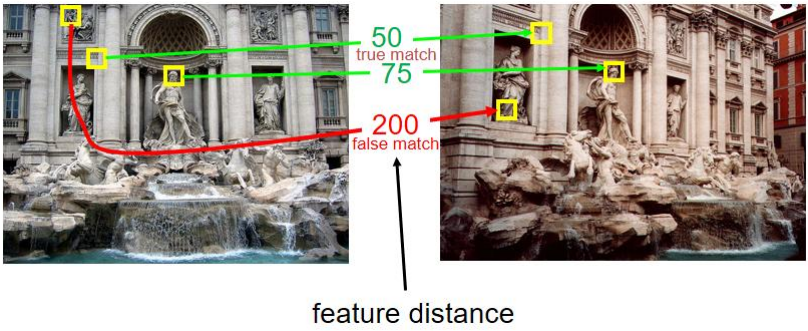
Evaluating the results

- How can we measure the performance of a feature matcher?

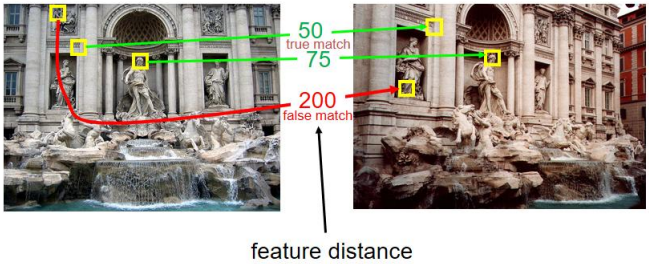


True/False positives

- The distance threshold affects performance
  - True positives = # of detected matches that survive the threshold that are correct
  - False positives = # of detected matches that survive the threshold that are incorrect



True/False positives: Robustness and Correctness



- We want to **maximize true positives**.
  - How do we set the threshold? (Note: we keep all matches with distance below the threshold.)
- We want to **minimize false positives**.
  - How do we set the threshold? (Note: we keep all matches with distance below the threshold.)
- **It is then all about having a robust distance metric and setting a good threshold.**

References

- "Computer Vision: Algorithms and Applications" by Szeliski (2nd edition).
- "Multiple View Geometry in Computer Vision" by Hartley and Zisserman.
- Prof. Avi Kak's Computer Vision Lectures at Purdue University.